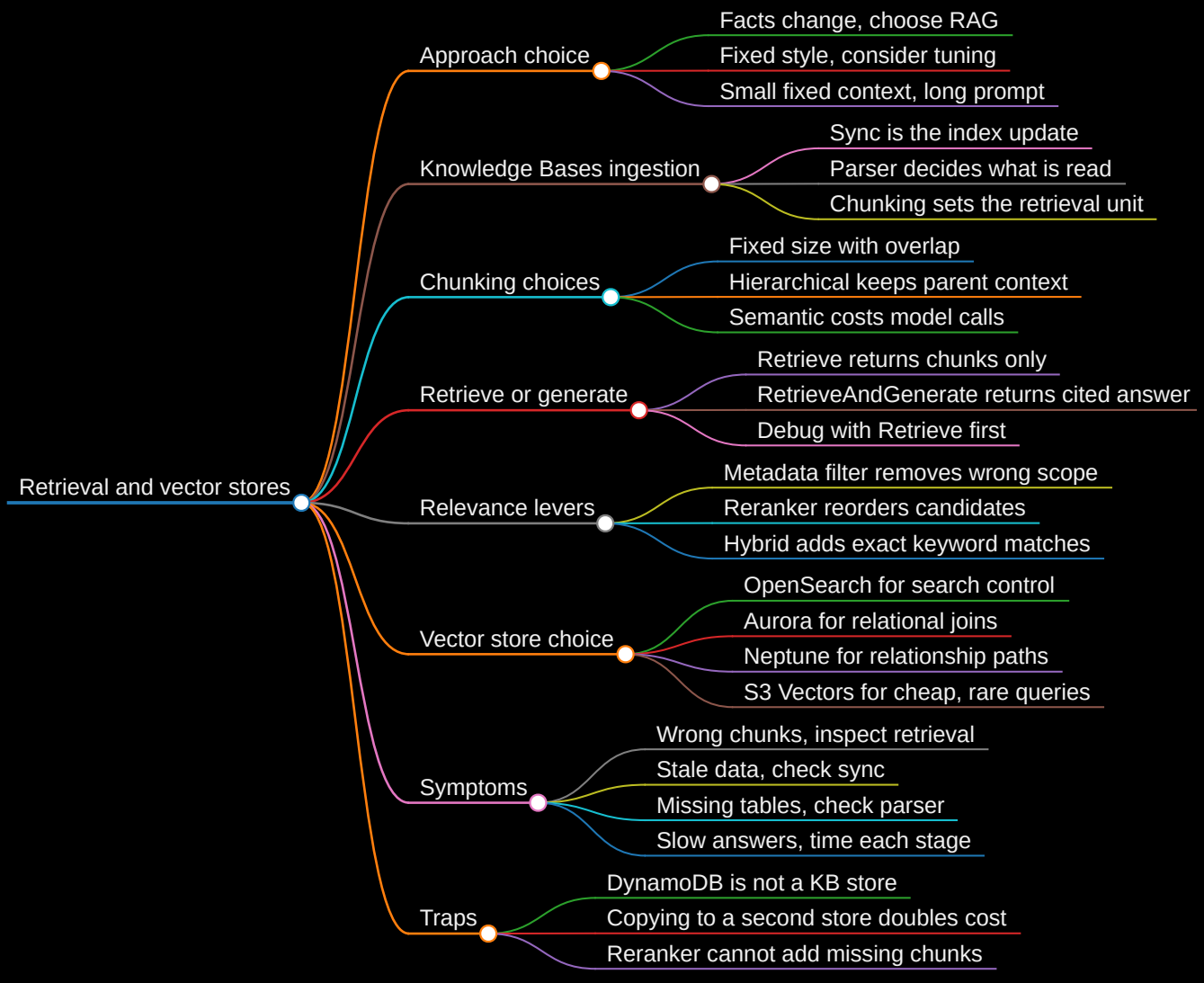
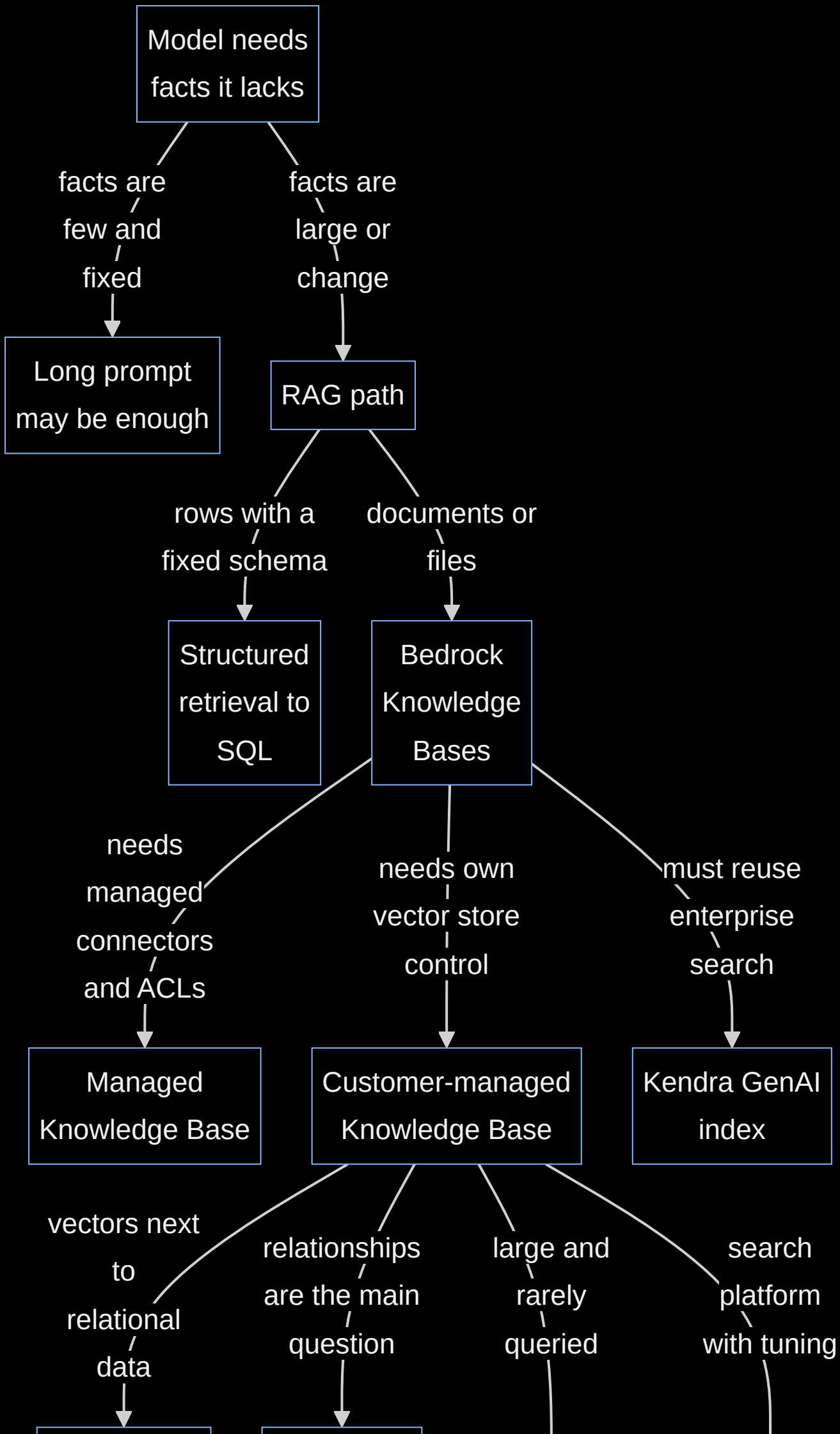
Retrieval and vector stores: Knowledge Bases, chunking, embeddings, and where vectors live

After this module, you can read a long retrieval scenario, find the constraint that decides the design, and reject each wrong option with one reason. You will separate retrieval from fine-tuning and long prompts. You will pick a Knowledge Bases setting and a vector store from the constraint, not from a service name you recognize.



The deciding constraint

Start from the scenario, not from the service list. First ask what the model lacks. Then ask what shape the missing facts have, because the shape decides the store. The tree below moves from the scenario to a service in a fixed order.


Model needs facts it lacks
facts are few and fixed
facts are large or change
Long prompt may be enough
RAG path
rows with a fixed schema
documents or files
Structured retrieval to SQL
Bedrock Knowledge Bases
needs managed connectors and ACLs
needs own vector store control
must reuse enterprise search
Managed Knowledge Base
Customer-managed Knowledge Base
Kendra GenAI index
vectors next to relational data
relationships are the main question
large and rarely queried
search platform with tuning

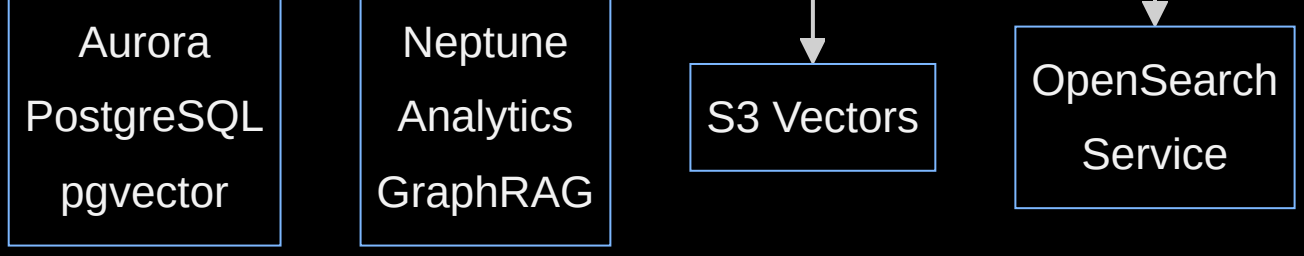


Figure: the first split is data shape, the second is permissions and control, and the third is the store.

Signal words in the scenario tell you which branch to take. Underline them in your head before you read the options. Each signal word below points to one branch.

- Rule: "few and fixed" facts point to a long prompt, not to retrieval.
- Rule: "rows", "columns", "total", and "count" point to structured retrieval.
- Rule: "existing document permissions" points to ACL-aware retrieval or Kendra.
- Rule: "who knows who" or "path between" points to a graph store.
- Rule: "exact codes" or "rare names" points to hybrid search.
- Rule: "rarely queried" plus "lowest cost" points to S3 Vectors.

Exam tip: a long scenario often lists five services and one constraint. The constraint is usually a number, a frequency, a permission, or a data shape. Find that first.

Services and features in scope

Retrieval augmented generation

RAG means the application fetches relevant source text at request time and adds it to the prompt. The model then answers from that text. AWS describes Knowledge Bases as a way to improve relevance and accuracy with this pattern, and to include citations so a reader can check the source.

Fact: RAG changes the facts the model sees, not the model weights. Updating a document updates the answer after the next sync.

Rule: choose RAG when facts change often, are too large for a prompt, or need source citations.

Trap: RAG does not change the tone or output format of a model. For a fixed format, a customization approach is the better fit.

Trap: a RAG system returns the wrong answer when retrieval returns the wrong chunks. A strong generator cannot repair missing evidence.

RAG pairs with a knowledge base, a vector store, a chunking strategy, and a reranker. The retrieval step is where most failures start.

Limits: the number of chunks you can pass depends on the model context window. Not in source for a specific token budget in the pages fetched.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base.html and course slides, pages 36 and 287.

Amazon Bedrock Knowledge Bases, managed and customer-managed

Knowledge Bases is the managed RAG feature in Bedrock. It has two types. A managed knowledge base runs ingestion, indexing, storage, and retrieval for you. A customer-managed knowledge base lets you choose and run the vector store, parsing, and ingestion settings.

Fact: managed knowledge bases offer connectors for S3, SharePoint, Confluence, Google Drive, OneDrive, and a web crawler. Document-level permission filtering with access control lists works with them, except for the web crawler.

Fact: third-party connectors and document-level permissions are listed as managed knowledge base features. Customer-managed bases need you to build those parts.

Rule: choose a managed knowledge base when the question needs SaaS connectors or ACL-aware retrieval with little setup.

Rule: choose a customer-managed knowledge base when the question names a specific vector store or demands your own parsing and index settings.

Trap: a managed knowledge base hides the storage choice. If the scenario requires Aurora or S3 Vectors by name, the managed type fails that constraint.

Exam tip: read whether the scenario says "managed" as a business need or as a Bedrock type. The overview separates both meanings.

The customer-managed type pairs with a vector store you create, an embedding model, and a service role that allows access to that store. The managed type pairs with Bedrock-selected models by default, with an option to choose your own.

Limits: the supported models and Regions page was not fetched. Regions for each feature: Not in source.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base.html

Data sources, sync, and ingestion jobs

A data source is the place where your files live. Ingestion reads the data source, parses it, chunks it, creates embeddings, and writes them to the vector store. Syncing starts an ingestion job. Each job reports status, and a finished job reports COMPLETE.

Fact: sync is incremental. Only added, modified, or deleted documents are processed since the previous sync. Deleted documents are removed from the vector store.

Fact: the API calls are StartIngestionJob, GetIngestionJob, ListIngestionJobs, and StopIngestionJob. A running job must exist before you stop it.

Fact: when only a metadata file changes, and the source is not CSV and no custom transformation Lambda is set, Bedrock updates metadata without calling the embedding model again.

Rule: when the question asks how to keep the index current with the least code, use a managed sync, triggered on a schedule or by an event.

Trap: uploading a new file to S3 does not change the index by itself. The index changes only after a sync or ingestion job runs.

Trap: a full rebuild with custom embedding code is not needed when the managed sync already handles adds, edits, and deletes.

Limits: for vector stores other than Aurora, new vectors can take a few minutes to become queryable after a job finishes. CSV files are always re-ingested when their metadata file changes.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/kb-data-source-sync-ingest.html and course slides, page 36.

Parsing options

Parsing turns raw files into text and structure before chunking. A parser that reads only text misses tables and images inside a PDF. The choice of parser therefore decides whether a table value can ever be retrieved.

Fact: the default parser reads text in txt, md, html, doc, docx, xls, xlsx, and pdf files. It has no usage charge.

Fact: Bedrock Data Automation and foundation model parsers process figures, charts, tables, and images in PDF, JPEG, and PNG files. They can store extracted images and tables as files in an S3 destination.

Fact: Bedrock Data Automation charges per page or per image. The foundation model parser charges by input and output tokens, lets you customize the extraction prompt, and accepts a total of at most 100 GB across all files.

Rule: choose a multimodal parser when the answer sits in a table, chart, or image inside a PDF.

Rule: choose the foundation model parser when the question needs a custom extraction instruction. Choose Bedrock Data Automation when no prompt is needed.

Trap: the default parser on a PDF of rate tables returns text only. The table values are not indexed as table content.

Trap: once a multimodal parser is selected, it applies to every PDF in the source, including text-only PDFs. Those files still incur parser charges.

Exam tip: the words "customize the extraction instruction" point to the foundation model parser. The words "fully managed with no prompting" point to Bedrock Data Automation.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/kb-advanced-parsing.html

Chunking strategies

Chunking splits a document into pieces. Each piece becomes one embedding and one retrievable unit. The chunk size sets how much text the model sees per match. A chunk that is too large dilutes the meaning. A chunk that is too small loses the context around the match.

Fact: fixed-size chunking takes a maximum token count per chunk and an overlap percentage between chunks. Default chunking targets about 300 tokens and keeps sentences whole. The course example uses 300 tokens with 20 percent overlap. The default overlap is Not in source.

Fact: hierarchical chunking takes a parent maximum size, a child maximum size, and an overlap in tokens. Retrieval matches child chunks and returns their parent chunks for more context.

Fact: semantic chunking takes a maximum token count, a buffer size, and a breakpoint percentile. A buffer of 1 combines the current sentence with one sentence before and one after. Semantic chunking calls a foundation model, so it has extra cost.

Fact: with no chunking, the whole document becomes one chunk. Custom chunking uses a transformation Lambda function that you write.

Fact: when a parsed page or table is 10,000 characters or fewer, retrieval can return the full page or table in place of the matched chunk. The number of results can then be lower than the number you asked for.

Rule: choose hierarchical chunking when small matches work but answers need the surrounding section.

Rule: choose semantic chunking when topic changes mark the right boundary and the extra model cost is acceptable.

Rule: choose custom Lambda chunking when the document structure is special and no native strategy fits.

Trap: hierarchical chunking is not recommended with S3 Vectors. Large parent and child token counts can exceed the metadata size limit per vector.

Trap: a chunk-size change does not fix a table that the parser never extracted. Check the parser first.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/kb-chunking.html and course slides, pages 42 to 44.

Embedding models and dimensions

An embedding model turns a chunk or a query into a vector. The vector has a fixed number of dimensions. The index in the vector store must use the same number. A mismatch causes a failure, so the model and the store must agree before you create the knowledge base.

Fact: the documented dimensions are Titan G1 Text 1,536, Titan V2 Text 1,024, 512, or 256, Cohere Embed English 1,024, and Cohere Embed Multilingual 1,024.

Fact: binary vectors are supported only in OpenSearch Serverless and OpenSearch managed clusters. S3 Vectors supports float32 only.

Fact: smaller vectors cost less to store and search, but retrieval quality can drop. The course slide describes this trade-off as a balance against domain fit.

Fact: binary vectors are described in the course slides as 32 times smaller than float32 vectors. Binary vectors are not embeddings in the strict sense and need a distance search that suits them.

Rule: pick the embedding model first, then pick a store that supports its dimension and vector type.

Trap: a question that asks for binary vectors and names S3 Vectors has a conflict. S3 Vectors does not store binary vectors.

Trap: the dimension in the index must match the model. Changing the model later usually means a new knowledge base, not a small edit.

Exam tip: find the number of dimensions in the scenario, then check the table. If no model matches, the store choice is wrong.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base-setup.html and course slides, pages 45 and 141.

Retrieve and RetrieveAndGenerate

Retrieve sends a query to a knowledge base and returns matching chunks. Each result carries content, a location, metadata, a score, and a document ID. RetrieveAndGenerate runs retrieval and then a model call, and returns an answer with citations.

Fact: Retrieve takes a retrieval query and a retrieval configuration. The vector configuration can hold numberOfResults, a filter, an override search type, and a reranking configuration. A managed knowledge base uses a managed search configuration instead.

Fact: Retrieve accepts user context. Bedrock uses it for access control filtering, so results include only documents the user can read.

Fact: the response field for results is retrievalResults. Each result has a score, so you can compare matches. Retrieve supports nextToken paging.

Limits: the default for numberOfResults is Not in source. The exact response shape of RetrieveAndGenerate was not fetched.

Rule: choose Retrieve when your code controls the prompt, changes the ranking, or must test retrieval alone.

Rule: choose RetrieveAndGenerate when a managed answer with citations is the whole requirement.

Trap: calling RetrieveAndGenerate to debug a bad answer hides the chunks. The model may still answer well from the wrong text.

Source: https://docs.aws.amazon.com/bedrock/latest/APIReference/API_agent-runtime_Retrieve.html and https://docs.aws.amazon.com/bedrock/latest/userguide/kb-chunking.html

Metadata filtering

Metadata filtering removes chunks that fail a rule before ranking. The rule uses keys and values attached to each source document. A filter does not read the text. It reads the attributes you stored at ingestion.

Fact: for S3 sources, attach metadata with a sidecar file. The file name is the source file name plus a .metadata.json suffix. A CSV file can use one column as content and other columns as metadata.

Fact: supported attribute types are STRING, NUMBER, BOOLEAN, and STRING_LIST. CSV metadata values are stored as strings.

Fact: the filter examples show equals, greaterThan, in, and startsWith conditions. Conditions combine with andAll and orAll. Each filter object in the examples holds one condition.

Fact: for OpenSearch custom fields, map the field as keyword, or as text with a keyword subfield. Otherwise filtering fails with a rewrite error.

Fact: on Aurora, metadata lives in a JSONB column with a GIN index. If a selective filter returns fewer results than expected, the docs recommend iterative index scans with pgvector 0.8.0 or later.

Rule: filter on attributes the user already has, such as region, product line, or effective date.

Trap: a filter cannot find a chunk that was never indexed. Add the metadata file, then sync again.

Trap: telling the model to ignore other regions in the prompt is not a filter. The chunks are still retrieved and sent.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/kb-metadata.html and https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base-setup.html

Reranking

A reranker model scores each candidate chunk against the query and reorders the list. It does not search the store. It works only on candidates that retrieval already returned.

Fact: reranking can run through the Rerank operation on any list of documents. It can also run inside Retrieve or RetrieveAndGenerate, and then its order replaces the default knowledge base order.

Fact: reranking works only on text data. Reranker models come from Amazon or Cohere, and availability is limited by Region in the course slides.

Fact: the Retrieve request holds a reranking configuration with a model ARN and a numberOfRerankedResults value.

Rule: retrieve a wider candidate set, then rerank down to a smaller set for the prompt.

Trap: a reranker cannot add a chunk that retrieval missed. If the right passage is absent, fix chunking or widen retrieval.

Trap: a reranker is a weaker fix for the wrong region than a metadata filter, because the wrong region chunk can still be ranked high.

Limits: reranker pricing and the supported Regions list were not fetched. Not in source for exact values.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/rerank.html and course slides, page 192.

Hybrid search

Hybrid search combines vector similarity with keyword matching. Vector search finds meaning. Keyword search finds exact tokens, such as codes, names, and rare words. The mix helps when the question contains both kinds of signal.

Fact: in OpenSearch, hybrid search needs filterable keyword fields for metadata. The course slides flag this as a required setup step.

Fact: on Aurora, hybrid search uses a full-text GIN index on the chunk text. The docs suggest an English dictionary for English content.

Fact: the vector configuration has an override search type field. Its enumerated values are Not in source in the pages fetched.

Rule: choose hybrid search when exact codes, SKUs, or rare names must match.

Trap: hybrid search widens the match, and a reranker orders the matches. They solve different problems, so one does not replace the other.

Trap: S3 Vectors has no native keyword search described in the pages fetched. For exact-match needs, the course slides point to a copy in OpenSearch, which adds cost.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base-setup.html and course slides, page 139.

Query decomposition and query rewriting

A single embedding of a multi-part question can average two topics. The result may match neither topic well. Decomposition splits the question into parts, and each part gets its own retrieval.

Fact: managed knowledge bases use agentic retrieval. The overview says it decomposes complex queries into sub-queries, retrieves across several knowledge bases, and checks whether the evidence is enough.

Fact: the course slides advise breaking up multi-part questions and reducing ambiguity before retrieval.

Rule: split a multi-part question when each part needs different evidence. Rewrite a follow-up question with its earlier turn before retrieval.

Trap: a follow-up such as "what about that one" embeds poorly on its own. The retrieval then returns unrelated chunks.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base.html and course slides, page 297.

Citations

A citation links a generated statement to the source that supports it. Citations are only as reliable as the retrieval result behind them. The source of truth is the retrieval location, not the text the model writes.

Fact: Knowledge Bases can include citations in generated responses, so the original source can be checked.

Fact: each retrieval result carries a location. The location types include S3, web, SharePoint, Confluence, Salesforce, Kendra document, and SQL.

Rule: attribution must come from retrieval location fields in the response, not from text the model writes about its own sources.

Trap: asking the model to list its sources in plain text is not traceable. The model can invent a source name.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base.html and https://docs.aws.amazon.com/bedrock/latest/APIReference/API_agent-runtime_Retrieve.html

Structured data retrieval to SQL

A knowledge base can connect to a structured data store, such as a table or a warehouse with a schema. The service converts the user question into a query for that store. The result is exact rows, not similar text.

Fact: Retrieve and RetrieveAndGenerate convert the user query into a query suitable for the connected store. The GenerateQuery operation converts natural language into SQL on its own, so you can place it in your own workflow.

Fact: the course slides describe a common pattern. A relational store holds structured records, and each record points to a larger object in S3.

Limits: the supported query engines and the permission setup were not fetched. Not in source.

Rule: choose structured retrieval when the answer is a count, a total, or a filter over rows with a schema.

Trap: vector search over row text returns passages that look similar. It does not calculate an exact total.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base-build-structured.html and course slides, page 154.

Amazon OpenSearch Serverless as a vector store

OpenSearch Serverless runs a vector search collection with no cluster to manage. The team creates an index with a vector field, a text field, and a Bedrock metadata field. Billing follows requests and stored data, not always-on nodes.

Fact: the index uses the faiss engine. The docs recommend Euclidean distance for float vectors. The course slides call it the default serverless choice for a quick development setup.

Fact: a vector collection can be private through a VPC endpoint. Binary vectors are supported.

Limits: serverless does not mean it scales to zero. Unused collections keep billing. Collection capacity limits: Not in source.

Rule: choose OpenSearch Serverless when traffic is uneven and the team wants low operations.

Trap: a serverless collection is a poor choice when the question demands shard and node tuning. Use a managed cluster for that.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base-setup.html and course slides, pages 139 and 660.

Amazon OpenSearch Service managed clusters

A managed cluster is a provisioned domain. You choose instance types and shards, and you pay for capacity while it runs. It gives more control and more things to manage.

Fact: for Knowledge Bases, the managed domain must have public network access. VPC-only domains are not supported.

Fact: binary vectors need engine version 2.16 or later. The k-NN index needs version 2.13 or later. The nmslib engine is not supported, so use faiss.

Fact: the course slides suggest shard sizes near 30 to 50 GB for semantic search and 10 to 30 GB for hybrid search. The slides also describe OpenSearch with an S3 Vectors engine, which works only on managed clusters.

Rule: choose a managed cluster when the question needs control over capacity, shards, or predictable latency.

Trap: a VPC-only domain is a common wrong choice. Knowledge Bases needs public access on that domain type.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base-setup.html and course slides, pages 142 and 149.

Amazon Aurora PostgreSQL with pgvector

Aurora PostgreSQL with pgvector stores vectors in a column next to ordinary relational data. Similarity search uses SQL operators. The vectors can take part in the same transaction as business rows.

Fact: the knowledge base table needs an id, an embedding vector, a text column, a metadata JSON column, and an optional custom metadata JSONB column. The embedding index uses hnsw with cosine operators.

Fact: the Aurora cluster must sit in the same account as the knowledge base.

Fact: the course slides say pgvector can compute cosine, L2, and inner product distances. It suits small and medium RAG systems with mostly structured data.

Rule: choose Aurora when vectors must join with, or commit with, relational business data.

Trap: an Aurora cluster in another account is not a valid setup for a knowledge base in the docs. Move the cluster or choose another store.

Trap: Aurora is a weak fit for a huge archive chosen only for storage cost. The course table notes that you run the tuning yourself.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base-setup.html and course slides, pages 159 and 657.

Neptune Analytics GraphRAG

Neptune Analytics is a graph analytics engine with a vector search index. GraphRAG uses the graph edges and the vector match together. The graph answers questions about paths and relationships, and the vectors find the starting points.

Fact: the vector index is created when the graph is created. Its dimension must match the embedding model.

Fact: a graph is sized in m-NCUs, and each m-NCU has about one GiB of memory. The capacity can change later.

Limits: the maximum number of hops and the graph size limits were not fetched. Not in source.

Rule: choose Neptune Analytics when the answer depends on paths between entities, such as ownership chains or dependencies.

Trap: a graph with no vector index does not do semantic search. A vector store alone does not do graph traversal.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base-setup.html and course slides, page 657.

Amazon S3 Vectors

S3 Vectors stores vectors in vector buckets. You create a vector index inside a bucket, write vectors with metadata, and query by similarity. It needs no infrastructure to provision. The design goal is low cost for large sets that are not queried often.

Fact: writes are strongly consistent. The docs describe latency of sub-second for infrequent queries and as low as 100 milliseconds for more frequent queries.

Fact: the dimension range is 1 to 4,096. The distance choices are cosine or Euclidean. Only float32 vectors are supported.

Fact: with Knowledge Bases, you can attach up to 1 KB of custom metadata and 35 metadata keys per vector. The metadata is filterable by default. Non-filterable keys are set on the index.

Fact: Block Public Access is always on for vector buckets. Access uses IAM with the s3vectors namespace.

Limits: the course slides state 10,000 indices per bucket and 2 billion vectors per index. The AWS limits page was not fetched, so the AWS value is Not in source.

Rule: choose S3 Vectors when the corpus is large, queries are rare, and sub-second latency is acceptable.

Trap: S3 Vectors cannot store binary vectors. A binary requirement points to OpenSearch.

Trap: copying S3 Vectors data into OpenSearch for speed means you pay for both stores. The course slides warn about this.

Source: https://docs.aws.amazon.com/AmazonS3/latest/userguide/s3-vectors.html and https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base-setup.html and course slides, pages 145 and 149.

MemoryDB

MemoryDB is a Redis-compatible durable in-memory database. The course slides say it now has vector capabilities. The Knowledge Bases store list in the pages fetched does not name MemoryDB.

Fact: the Knowledge Bases store list in the setup page names OpenSearch Serverless, OpenSearch managed clusters, Aurora, Neptune Analytics, S3 Vectors, Pinecone, Redis Enterprise Cloud, and MongoDB Atlas.

Limits: MemoryDB support as a Knowledge Bases store is Not in source.

Rule: for a Knowledge Bases question, choose only a store from the listed set. For a general low-latency vector need, MemoryDB is a separate design question.

Trap: MemoryDB and Redis Enterprise Cloud are different products. The listed Redis option does not make MemoryDB a valid knowledge base store.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base-setup.html and course slides, page 37.

DynamoDB

DynamoDB is an operational key-value and document database. It is not on the Knowledge Bases store list. The course slides describe a newer vector index feature on DynamoDB, which is a separate capability from a Knowledge Bases store.

Fact: the course slides describe a SearchVectors operation with approximate nearest neighbor search. It is on-demand capacity only, returns at most 16 MB with no pagination, and filters only on equality.

Fact: the course slides describe a zero-ETL pipeline from DynamoDB to OpenSearch, which you can then use as a Knowledge Bases store. The AWS page for the vector index feature was not fetched. Its current status is Not in source.

Rule: when DynamoDB is the requested Knowledge Bases store, reject it. Keep DynamoDB as the source and sync to a supported store.

Trap: a DynamoDB scan with a similarity check in code is not vector search. It reads every item and does not rank by meaning.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base-setup.html and course slides, pages 186 to 188.

Kendra as a retriever

Kendra is an enterprise search service with document permission handling. The Knowledge Bases docs include a Kendra GenAI index option. Kendra is the right retriever when existing document permissions must decide what a user sees.

Fact: the Knowledge Bases overview lists a build path with a Kendra GenAI index. Retrieval results can carry a Kendra document location.

Fact: the course slides say Kendra fits document-level access control with SharePoint and Confluence. The slides also say its cost can be high for large deployments.

Limits: Kendra query pricing and index size limits were not fetched. Not in source.

Rule: choose Kendra, or a managed knowledge base with ACL filtering, when existing document permissions must hold at retrieval time.

Trap: a metadata field named owner does not enforce permissions by itself. The application must set the filter from a trusted identity.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base.html and https://docs.aws.amazon.com/bedrock/latest/APIReference/API_agent-runtime_Retrieve.html and course slides, pages 657 to 658.

X versus Y: confusable pairs

Each pair below has one decision tree, a three-line comparison, one question, and one trap. Use the question to decide, then check the trap against the distractors in the options.

RAG versus fine-tuning

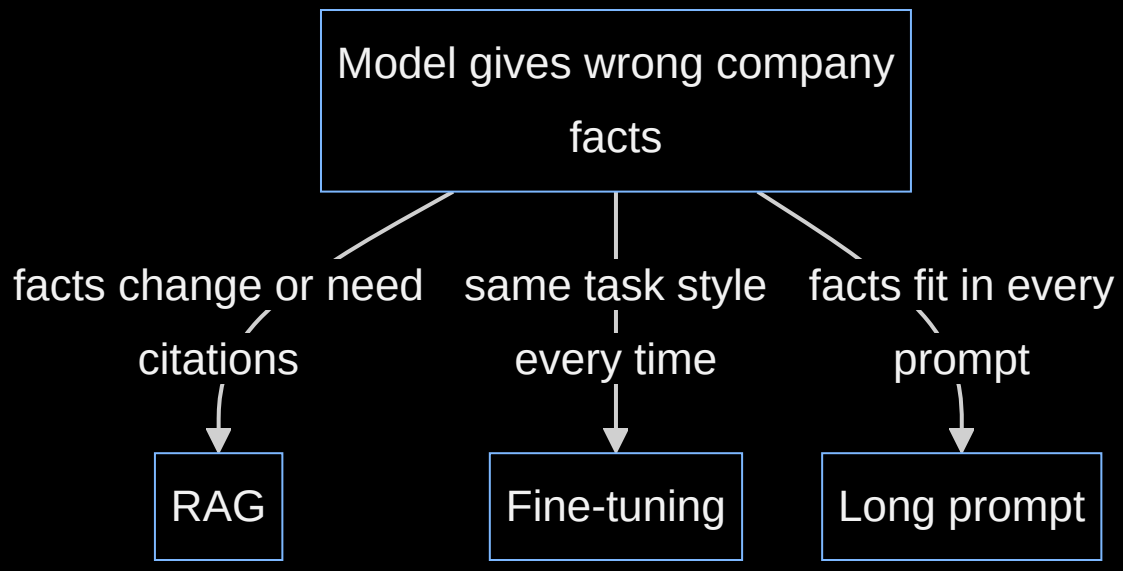


Figure: change in facts points to retrieval, change in behavior points to tuning.

- Purpose: RAG supplies facts at request time. Fine-tuning changes how the model behaves.
- Input: RAG takes a query and an index. Fine-tuning takes labeled training examples and a training job.
- Output or effect: RAG returns an answer with source spans. Fine-tuning bakes old facts in until you train again.

The one question to ask yourself is whether the answer must change when a source document changes. If yes, the fix is retrieval. Fine-tuning can teach a format or a tone, but it does not give a per-answer source.

Trap: fine-tuning on policy text to make the model "know" new rules. The answer has no citation, and each policy edit needs another training run.

RAG versus long prompts

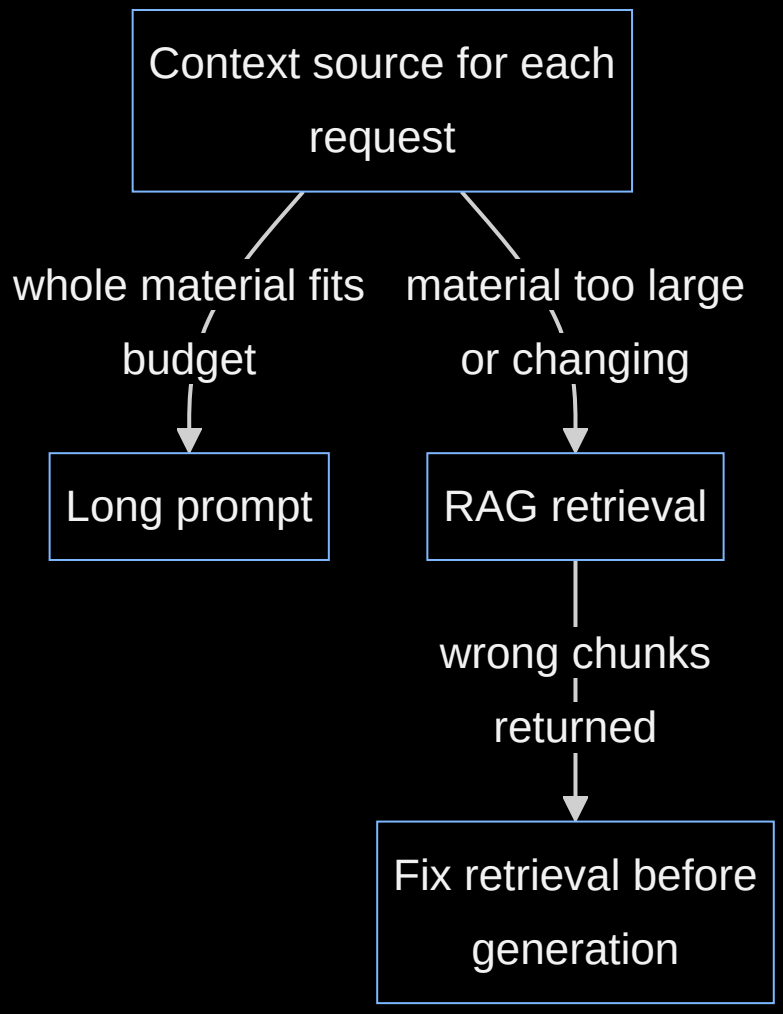


Figure: the size of the material and its change rate decide the approach.

- Purpose: a long prompt sends all material each call. RAG sends only the matched chunks.
- Input: a long prompt needs the full text in the context window. RAG needs an index and a query.
- Output or effect: a long prompt pays for every token on every call. RAG pays less per call but depends on retrieval quality.

The one question is whether the full material fits the budget on every call. If it does and it rarely changes, a prompt can be simpler and safer.

Trap: a long prompt is not always the cheap choice. When the material grows or changes often, the per-call token cost grows with it.

Retrieve versus RetrieveAndGenerate

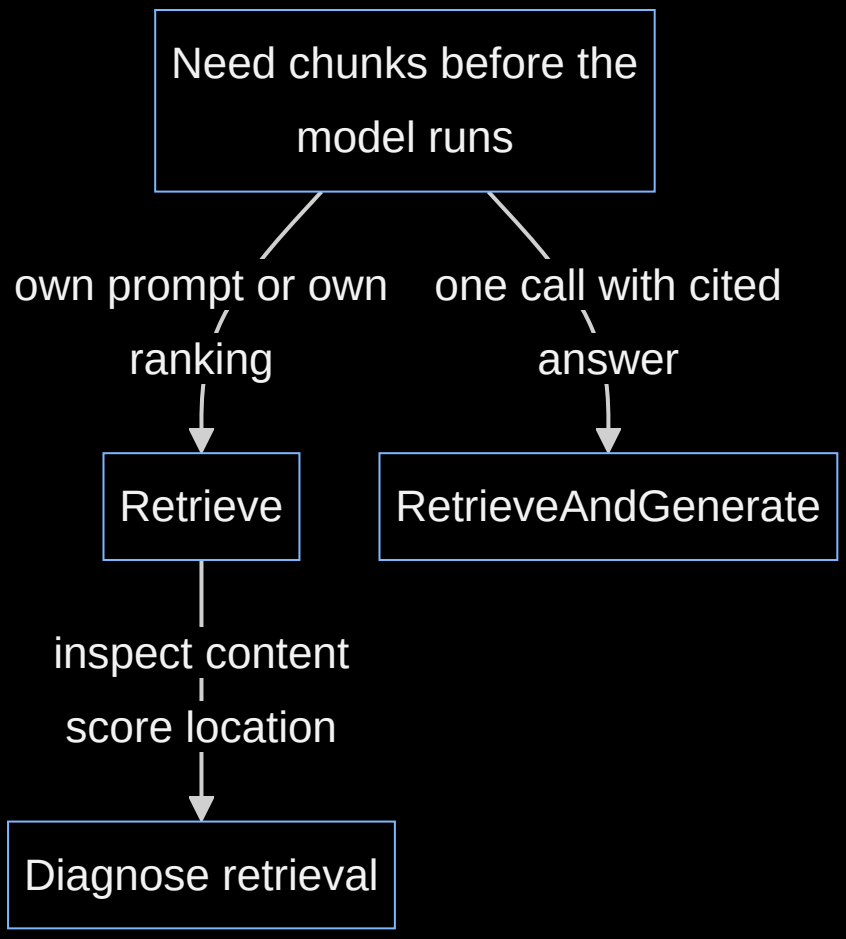


Figure: Retrieve gives control, RetrieveAndGenerate gives a managed answer.

- Purpose: Retrieve returns evidence. RetrieveAndGenerate returns evidence plus a generated answer.
- Input: Retrieve takes a query and a configuration. RetrieveAndGenerate takes the query and the generation settings.
- Output or effect: Retrieve returns results with scores. RetrieveAndGenerate returns text with citations.

The one question is whether your code must see or change the chunks before the model runs. If yes, call Retrieve. If the managed answer with citations is enough, use RetrieveAndGenerate.

Trap: using RetrieveAndGenerate to find out why an answer is wrong. You cannot see which chunks drove the error.

Hierarchical versus semantic chunking

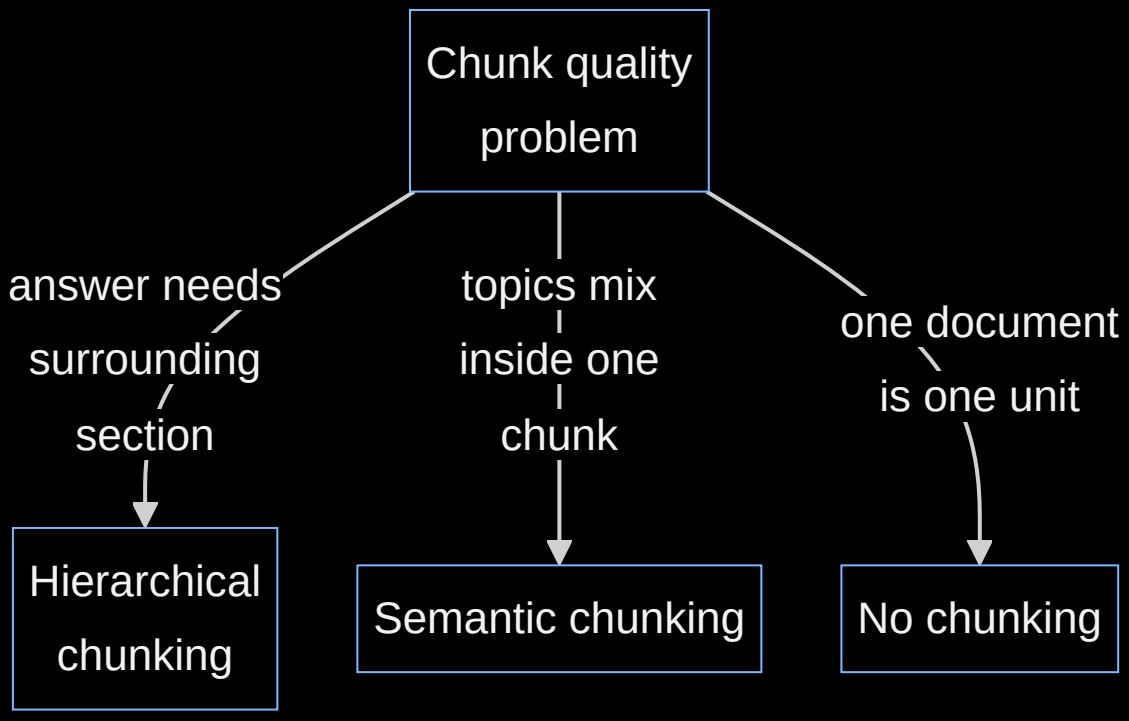


Figure: the failure type decides the strategy, not the size of the document.

- Purpose: hierarchical keeps small matches with parent context. Semantic places boundaries at topic changes.
- Input: hierarchical takes parent size, child size, and overlap. Semantic takes maximum tokens, buffer size, and breakpoint percentile.
- Output or effect: hierarchical can return fewer results than requested. Semantic makes variable chunks and calls a foundation model.

The one question is whether the answer lacks surrounding context, or whether one chunk mixes two topics. Those are different failures with different fixes.

Trap: semantic chunking does not recover a table that the parser never extracted. That is a parser problem.

Default parser versus multimodal parsers

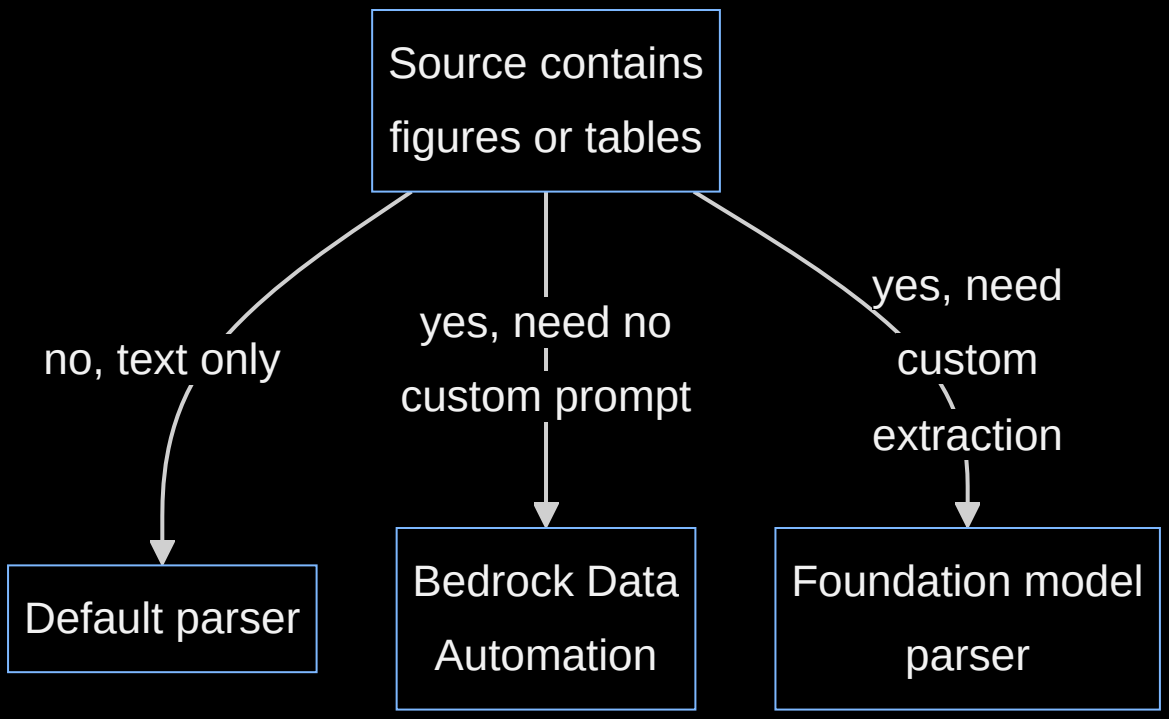


Figure: the content type and the need for a custom instruction decide the parser.

- Purpose: the default parser reads text. The multimodal parsers also read figures, charts, tables, and images.
- Input: the default parser takes text file types. The multimodal parsers take PDF, JPEG, and PNG files.
- Output or effect: the default parser returns text only. The multimodal parsers return text plus extracted visual elements stored in S3.

The one question is whether the answer lives inside a table, chart, or image. If it does, the default parser cannot supply it.

Trap: a multimodal parser on a text-only PDF set still incurs parser charges. Choose the parser by content, not by habit.

Metadata filter versus reranker

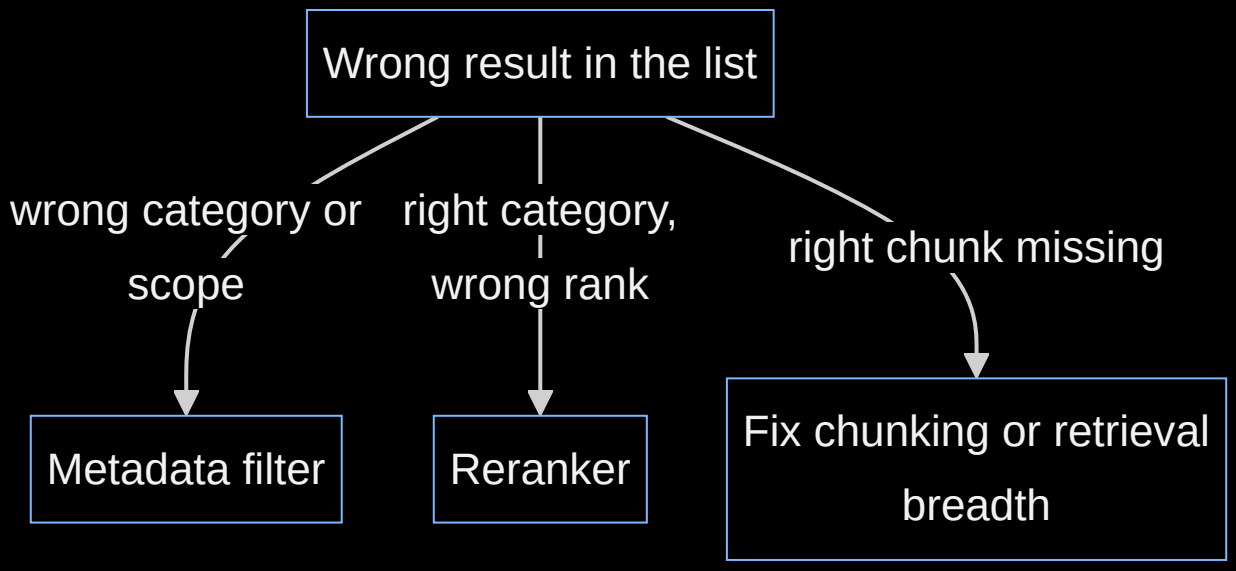


Figure: a filter removes candidates, a reranker reorders them, and neither adds missing text.

- Purpose: a filter removes candidates that fail a rule. A reranker orders the candidates by relevance.
- Input: a filter uses metadata keys and values. A reranker uses the query and candidate text.
- Output or effect: a filter shrinks the set. A reranker keeps the set and changes the order.

The one question is whether the wrong result belongs to the wrong category, or the right category sits too low. A category problem needs a filter.

Trap: a reranker is not a scope control. It can still place a wrong-region chunk high if the words match.

Hybrid search versus semantic-only search

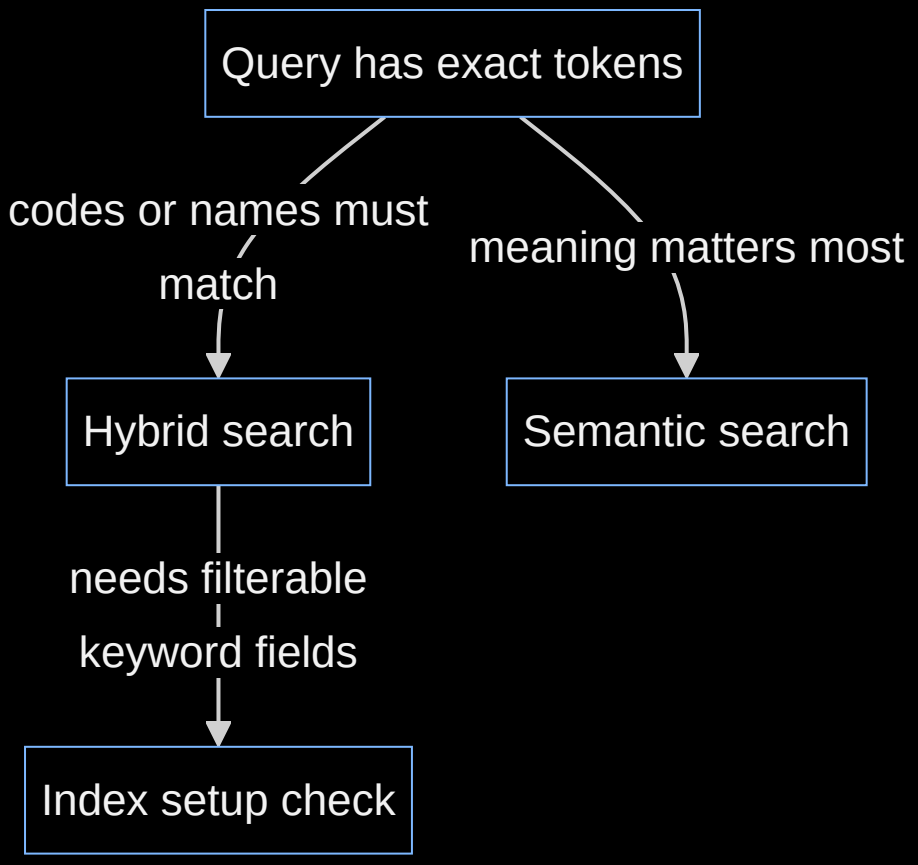


Figure: exact tokens in the question point to a keyword component.

- Purpose: hybrid adds keyword matching to vector similarity. Semantic search uses vector similarity only.
- Input: hybrid takes the query text and its vector. Semantic takes the vector.
- Output or effect: hybrid can rank exact matches higher. Semantic can miss a code that has no close meaning.

The one question is whether a missing exact match would make the answer wrong. For codes, names, and rare terms, the answer is yes.

Trap: a semantic-only index on SKU codes returns products that sound alike. Exact codes need the keyword part.

OpenSearch Serverless versus managed OpenSearch

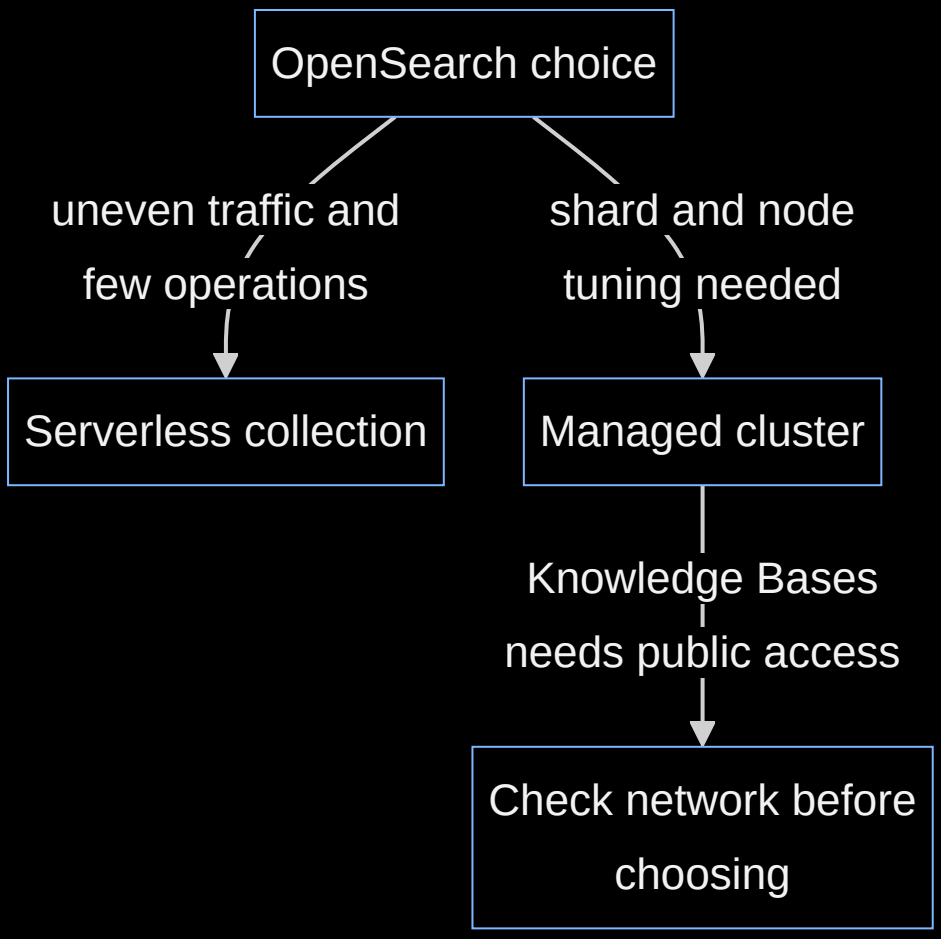


Figure: operations effort and tuning control separate the two options.

- Purpose: both store and search vectors. Serverless removes node management. Managed gives full control.
- Input: serverless takes a collection and an index. Managed takes a domain with instance and shard settings.
- Output or effect: serverless bills per request and storage, with variable latency. Managed bills for running capacity, with more predictable latency.

The one question is whether the team must tune shards and nodes, or accept variable latency to avoid that work.

Trap: choosing a managed cluster for uneven traffic and a small ops team. The always-on capacity cost then fits the traffic badly.

Aurora pgvector versus OpenSearch

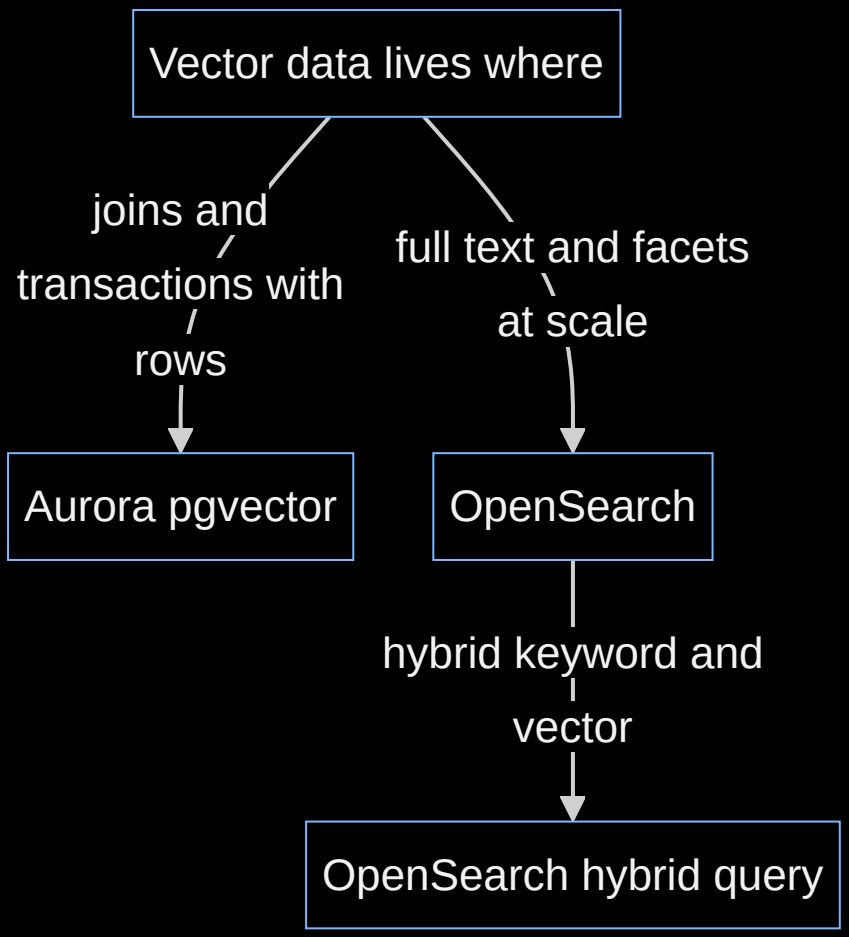


Figure: relational work points to Aurora, search engine work points to OpenSearch.

- Purpose: Aurora keeps vectors with relational data. OpenSearch is a search engine with vector support.
- Input: Aurora takes rows with an embedding column and SQL. OpenSearch takes documents with mappings.
- Output or effect: Aurora returns SQL joins and transactional consistency. OpenSearch returns hybrid and faceted search.

The one question is whether the vectors must join or commit with business rows. If yes, Aurora fits.

Trap: Aurora as the choice for a large keyword search platform. The search features and tuning control sit in OpenSearch.

S3 Vectors versus OpenSearch

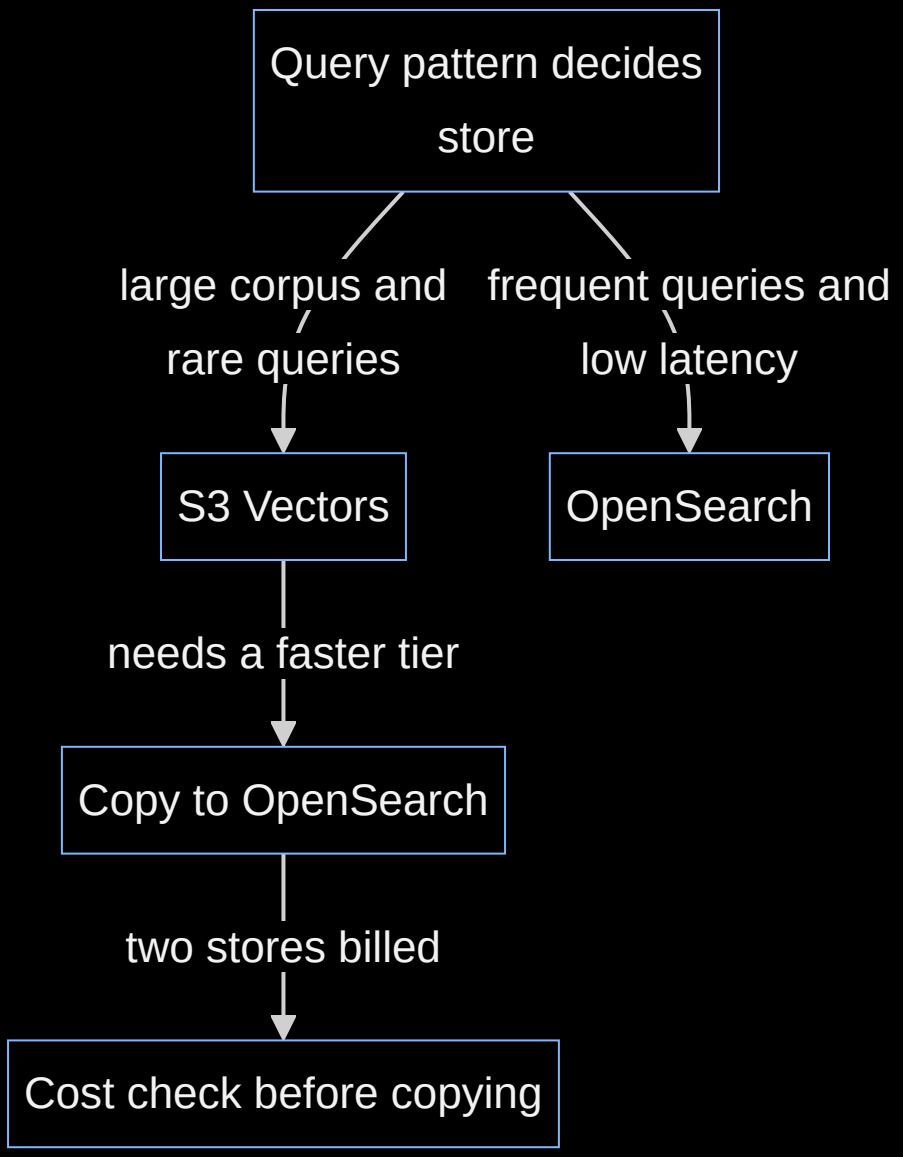


Figure: rare queries favor S3 Vectors, frequent low-latency queries favor OpenSearch.

- Purpose: S3 Vectors stores large sets cheaply. OpenSearch gives fast, rich search.
- Input: S3 Vectors takes a vector bucket, an index, and metadata. OpenSearch takes an index with mappings.
- Output or effect: S3 Vectors gives sub-second to about 100 ms latency. OpenSearch gives predictable, faster response at higher cost.

The one question is how often the store is queried, and how fast the answer must arrive.

Trap: a tiered design with S3 Vectors for storage and OpenSearch for speed doubles the stored data cost. Check the bill before choosing it.

Neptune Analytics versus Aurora pgvector

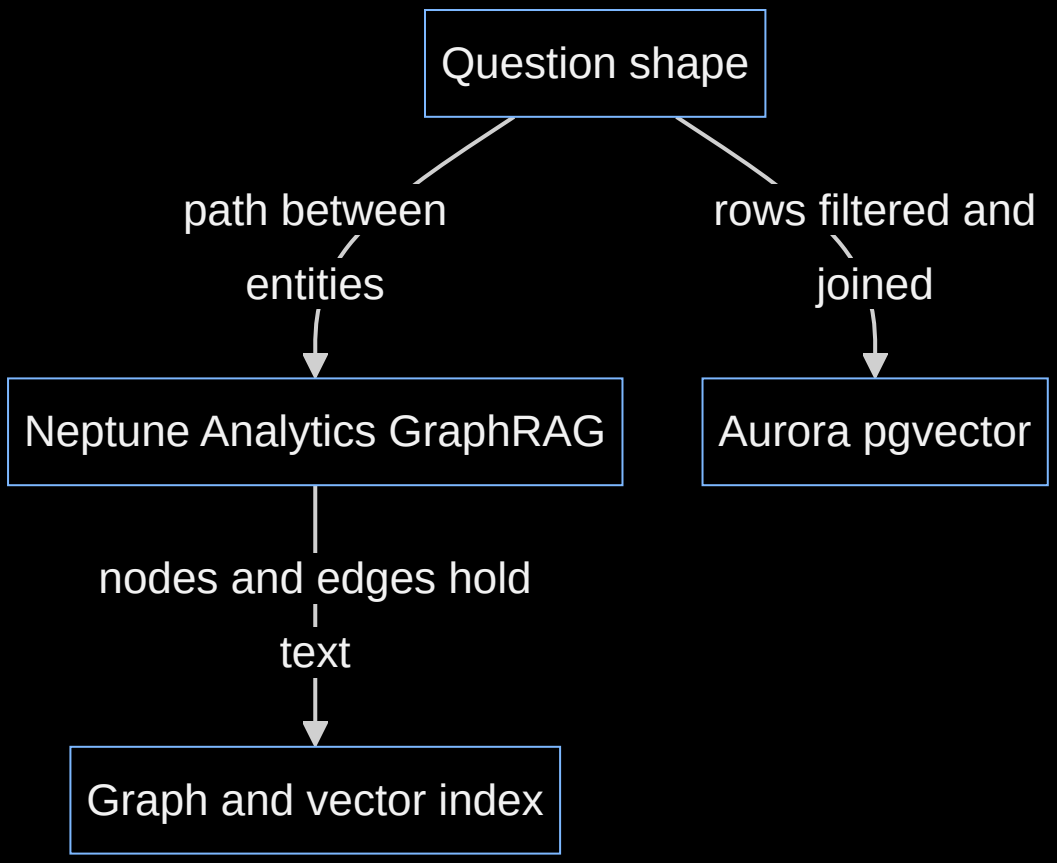


Figure: the answer shape, a path or a row set, decides the store.

- Purpose: Neptune traverses relationships. Aurora filters and joins rows.
- Input: Neptune takes nodes, edges, and a vector index. Aurora takes tables and a vector column.
- Output or effect: Neptune answers multi-hop paths. Aurora answers row sets and joins.

The one question is whether the answer is a chain of relationships between entities.

Trap: a relational join is not graph traversal. A query that follows unknown numbers of hops needs a graph store.

Kendra versus a knowledge base with ACL filtering

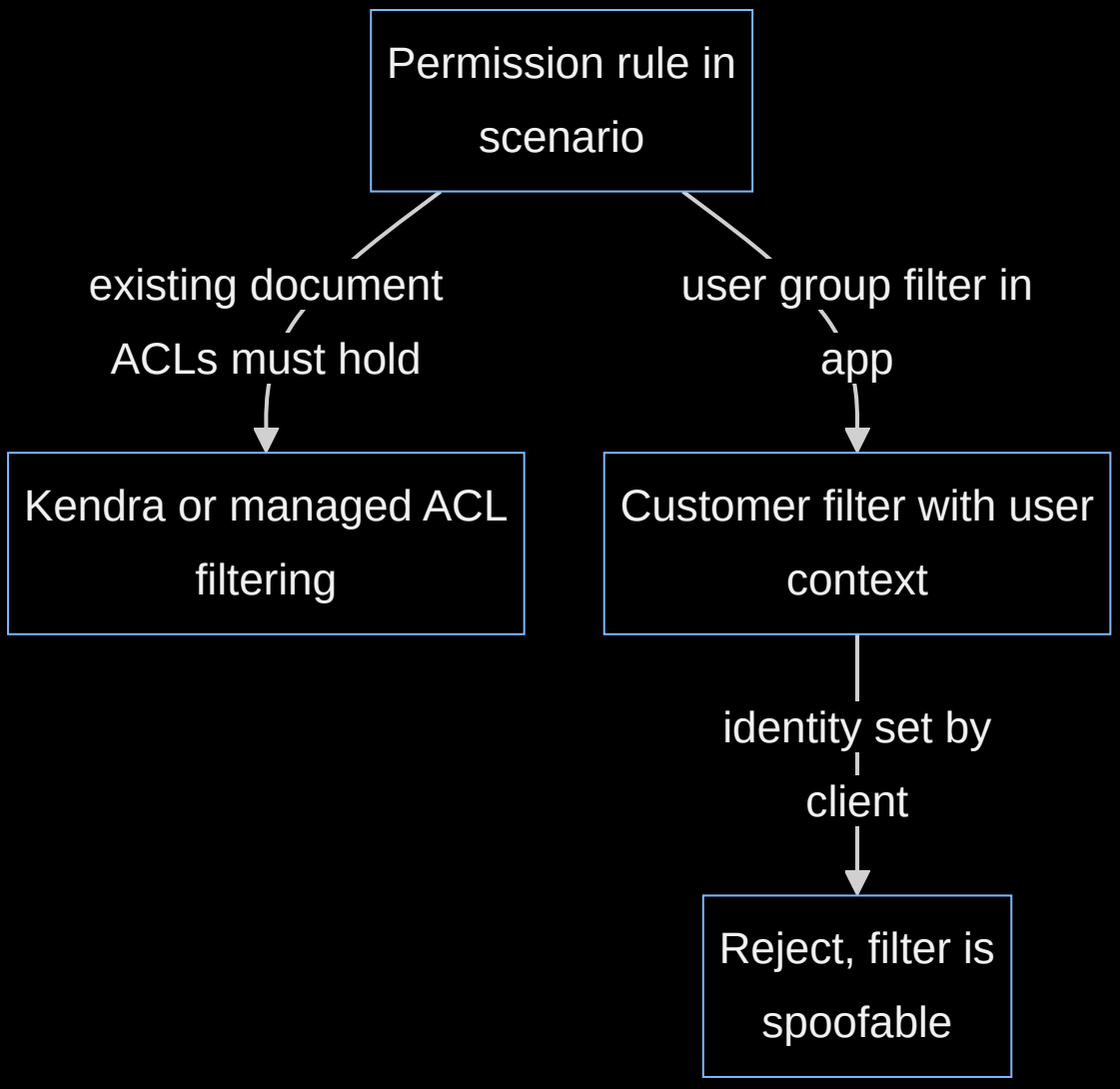


Figure: permissions must come from a trusted identity, not from a client value.

- Purpose: both enforce which documents a user may see at retrieval time.
- Input: Kendra and managed ACL filtering read document permissions. A customer-managed base needs a filter built from identity.
- Output or effect: results contain only documents the user can read.

The one question is whether existing document permissions must decide each chunk.

Trap: a filter that uses a user ID sent by the browser is not a security control. The server must set it from a trusted identity.

Vector search versus structured SQL retrieval

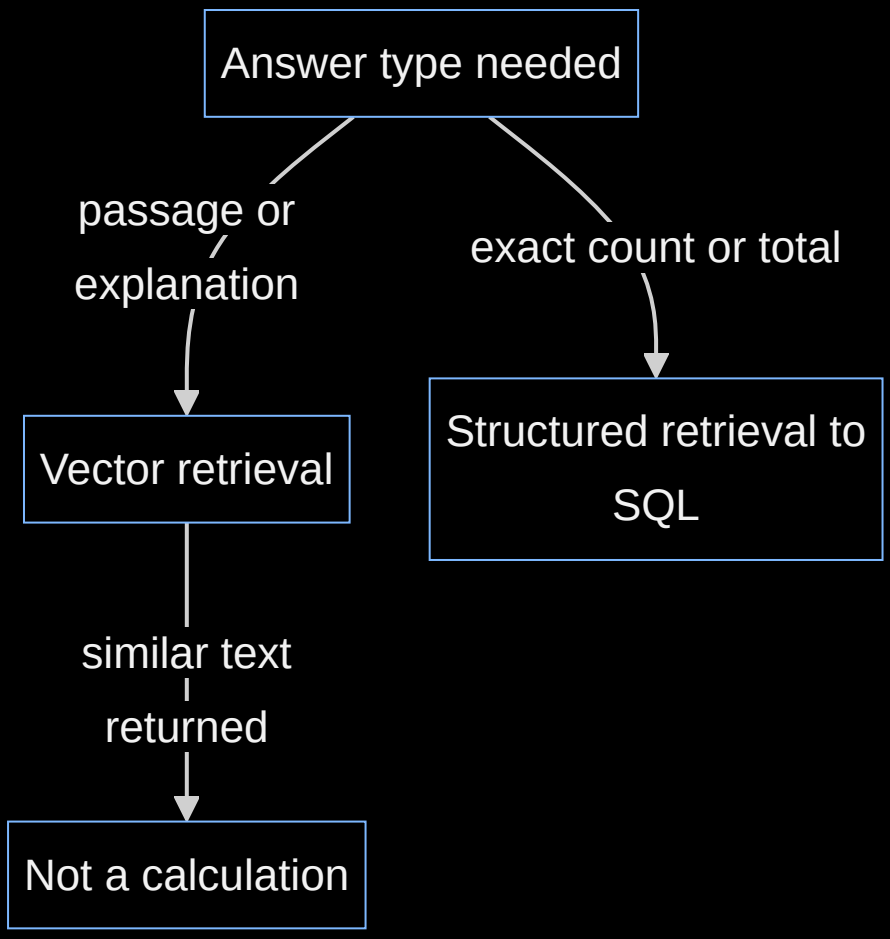


Figure: passages come from similarity, totals come from a query over rows.

- Purpose: vector retrieval finds similar text. SQL retrieval computes exact results over rows.
- Input: vector retrieval takes a query embedding. SQL retrieval takes a natural language question against a schema.
- Output or effect: vector retrieval returns passages with scores. SQL retrieval returns rows or aggregates.

The one question is whether the answer is a passage or an exact calculation.

Trap: a vector index of product rows returns similar products. It does not give the correct sum for a region.

DynamoDB versus a supported vector store

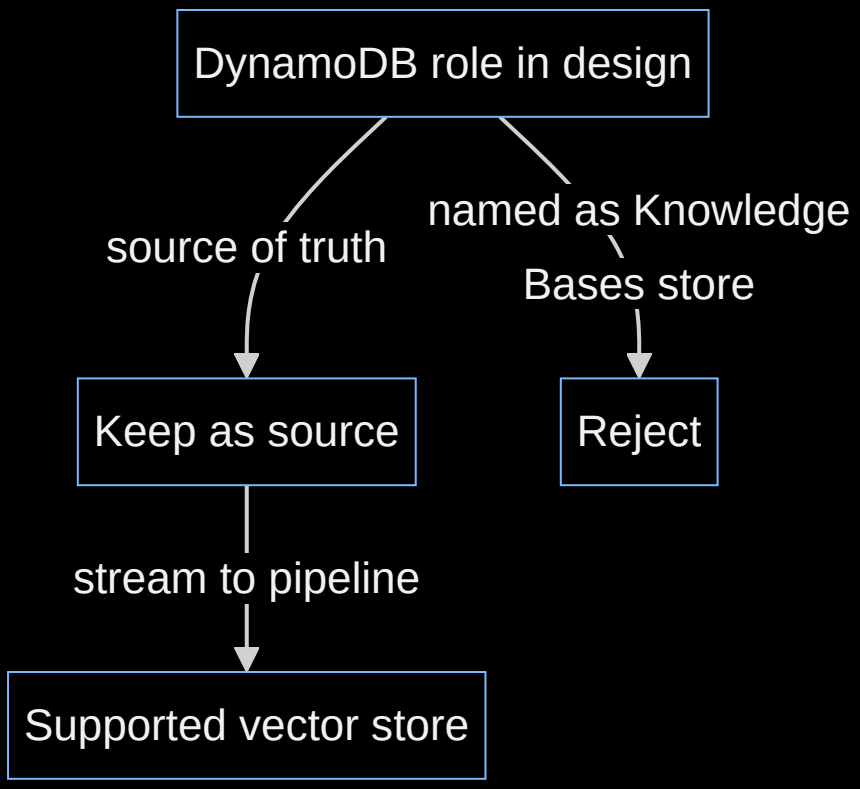


Figure: DynamoDB can feed a store, but it is not the listed store in this design.

- Purpose: DynamoDB stores operational items. A Knowledge Bases store indexes vectors for retrieval.
- Input: DynamoDB takes items and streams. A Knowledge Bases store takes embeddings from the ingestion job.
- Output or effect: DynamoDB returns items by key. A supported store returns ranked chunks.

The one question is whether the named store is on the Knowledge Bases list.

Trap: a stream from DynamoDB to Lambda that writes embeddings to OpenSearch is valid. Naming DynamoDB as the Knowledge Bases store itself is not.

Read a long question

Each worked scenario below uses three sentences. Underline the constraint words, eliminate each wrong option with one reason, then check the answer.

Worked scenario one: contracts with rate tables

Scenario: A legal team stores contracts as PDFs with rate tables. Users ask for one rate and need the source page, but only for their own region. The team wants managed generation with citations and no model training.

Constraint words: **rate tables**, **source page**, **their own region**, **citations**, **no model training**.

- Trap: fine-tune a model on the contracts. It fails "no model training" and gives no page citation.
- Trap: the default parser on the PDFs. It keeps text only, so the rate tables are lost.
- Trap: store contracts as DynamoDB items and scan by region. DynamoDB is not a Knowledge Bases store, and a scan does not rank by meaning.

Rule: answer is a Knowledge Base with a multimodal parser, a region metadata filter, and RetrieveAndGenerate.

Worked scenario two: a support site that changes hourly

Scenario: A product support site changes every hour. The assistant must answer with the newest article and must match exact model codes. The team also wants no custom sync code.

Constraint words: **every hour**, **newest article**, **exact model codes**, **no custom sync code**.

- Trap: place all articles in every prompt. The set grows past the budget, and every call pays for all of it.
- Trap: rebuild the whole index weekly with a script. It misses hourly change and adds custom code.
- Trap: vector search only on S3 Vectors. Exact codes need keyword matching, and the pages fetched show no native keyword search there.

Rule: answer is a Knowledge Base on OpenSearch Serverless with hybrid search, and a scheduled ingestion job that runs every hour.

Worked scenario three: a rarely queried research archive

Scenario: A research group keeps 40 million vectors from archived papers. Analysts run a few queries each month and accept a delay of up to one second. The budget is the first priority.

Constraint words: **few queries each month**, **accept a delay**, **budget first**.

- Trap: an always-on OpenSearch managed cluster. It bills for capacity that sits idle between monthly queries.
- Trap: an always-on Aurora cluster. The scenario has no need for SQL joins or transactions, so the database adds cost with no matching need.
- Trap: a Kendra index. The scenario names no permission rule, and the course slides flag Kendra cost at scale.

Rule: answer is an S3 Vectors index as the Knowledge Base store, with the dimension matching the chosen model.

Troubleshooting RAG by symptom

Start from the symptom, then check the stage that produces it. The retrieval stage is checked before the generation stage. Work in this order each time.

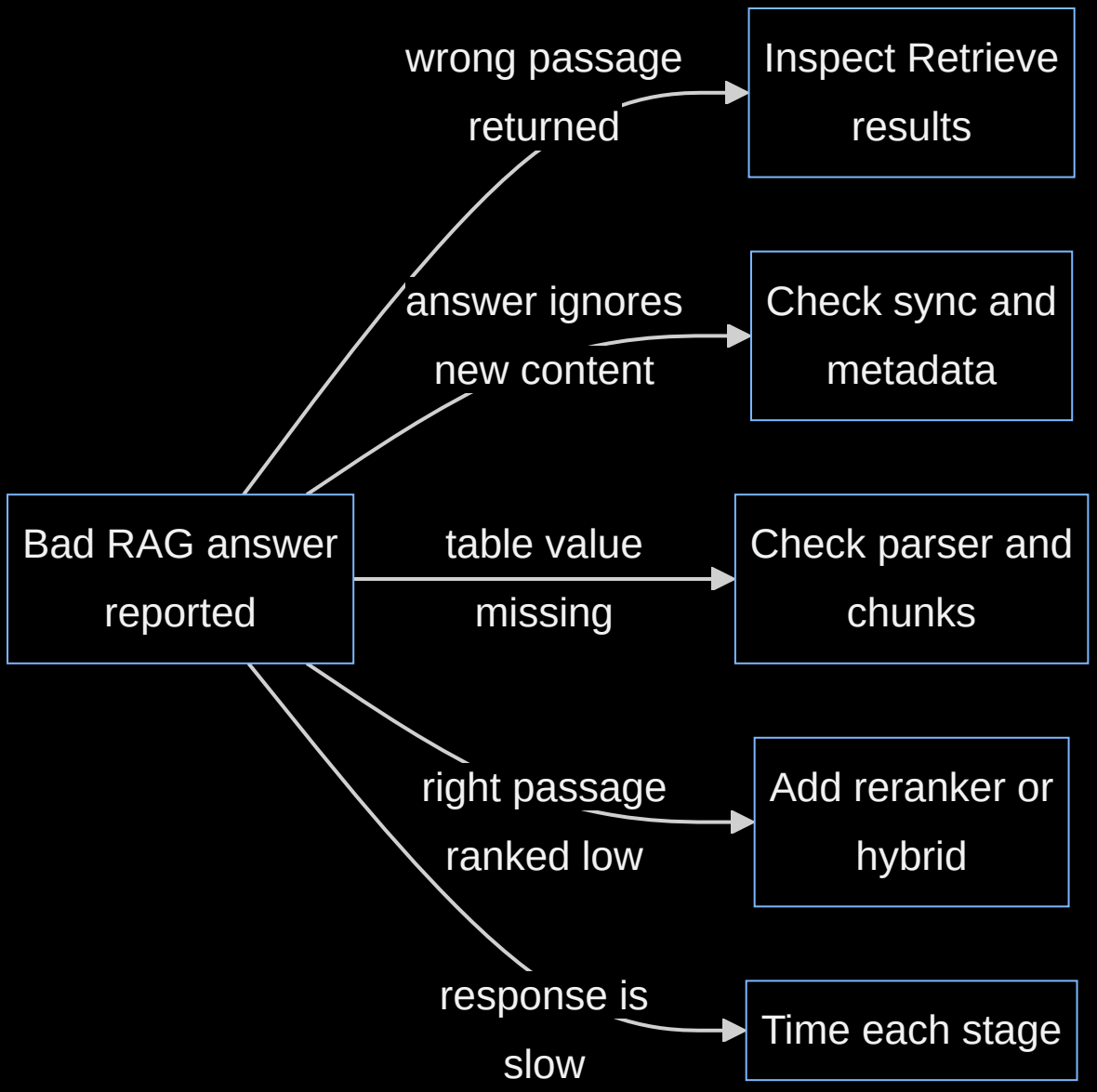


Figure: each symptom maps to one stage, and that stage is checked first.

Symptom: wrong chunks

Procedure: Call Retrieve alone with the same question. Read the content, location, and score.

Fact: a wrong chunk can come from the chunking strategy, the embedding model, or a missing filter. The retrieval output shows which one is likely.

Rule: if the top results are wrong, change retrieval or chunking before you change the generator.

Trap: switching to a larger generation model when retrieval returns the wrong passage. The larger model still sees the wrong text.

Source: https://docs.aws.amazon.com/bedrock/latest/APIReference/API_agent-runtime_Retrieve.html

Symptom: stale data

Procedure: Confirm the last ingestion job reached COMPLETE. Then run Retrieve again.

Fact: sync is incremental and processes only changed documents. Non-Aurora stores can need a few minutes after the job finishes.

Rule: if the source changed and no sync ran, the fix is a sync, not a prompt edit.

Trap: a prompt that says "use the newest version" does not refresh the index.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/kb-data-source-sync-ingest.html

Symptom: missing tables

Procedure: Check the parser setting. Then inspect one chunk that should hold the table.

Fact: the default parser keeps text only. A multimodal parser extracts tables and figures from PDFs.

Fact: a table or page of 10,000 characters or fewer can return with full context. A table that is never parsed cannot return at all.

Rule: if the table is absent from the chunks, change the parser before the chunk size.

Trap: increasing numberOfResults does not add a table that was never parsed.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/kb-advanced-parsing.html and https://docs.aws.amazon.com/bedrock/latest/userguide/kb-chunking.html

Symptom: irrelevant results

Procedure: Add a metadata filter for the scope. Compare results with and without it.

Fact: a filter removes candidates by attribute. Hybrid search adds exact keyword matching. A reranker reorders the candidates.

Rule: a wrong category needs a filter, a wrong order needs a reranker, and a missing exact term needs hybrid search.

Trap: adding more chunks to the prompt when the results are irrelevant. More noise does not add relevance.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/rerank.html and https://docs.aws.amazon.com/bedrock/latest/userguide/kb-metadata.html

Symptom: latency

Procedure: Time Retrieve and generation as separate steps before changing any model.

Fact: a reranker adds a model call. RetrieveAndGenerate adds generation time. Semantic chunking adds cost at ingestion, not at query time.

Fact: S3 Vectors is designed for sub-second latency on infrequent queries. OpenSearch managed clusters use provisioned capacity, and Serverless latency can vary.

Rule: if retrieval is the slow step, change the store or the candidate count. If generation is slow, change the generator or the output length.

Trap: moving to a larger model when retrieval is the slow stage. The delay stays in the search.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/rerank.html and https://docs.aws.amazon.com/AmazonS3/latest/userguide/s3-vectors.html

---

Distractor patterns seen in the tests

These patterns repeat across long questions. Each one looks correct at first read. Each one fails for a concept reason, not a memorized fact.

Trap: right store, wrong scale of control. A serverless option is offered for a team that needs shard tuning. The control need decides, not the store name.

Trap: right store, manual pipeline. A custom re-embedding job is offered where a managed sync already handles changes. More code adds failure points.

Trap: a fix at the wrong stage. A reranker or larger prompt is offered for missing chunks. The stage that produced the gap must change.

Trap: model change instead of retrieval fix. Fine-tuning or a bigger model is offered for a facts problem. Facts live in the index, not in the weights.

Trap: transactional store named as a vector store. DynamoDB or another operational database is offered as the Knowledge Bases store. The listed store set decides.

Trap: two stores billed for one job. A copy into a faster store is offered without a cost check. The data cost doubles.

Trap: network setting that breaks the setup. A private-only domain is offered for a Knowledge Base. The docs require public access for managed domains.

Trap: security by instruction. A prompt that says to hide other tenants is offered instead of a filter or ACL. Instructions do not stop retrieval of the wrong chunks.

Trap: cross-account mismatch. An Aurora cluster in another account is offered for a Knowledge Base. The docs require the same account.

Drill

Each prompt is one line. Read the answer on the next line, then say why the wrong option fails.

Policy PDFs hold the answers in tables. Which parser setting comes first?

Rule: a multimodal parser, because the default parser keeps text only.

Source files change daily, and users still get an answer from an older version. What runs first?

Rule: a sync or ingestion job, so added, changed, and deleted files reach the index.

A reranker is on, but the right passage never appears in the candidates. What is wrong?

Trap: the reranker only reorders candidates. Widen retrieval or fix chunking.

Users may see only documents their group can read. What must hold at retrieval time?

Rule: access control filtering from a trusted identity, or a Kendra or managed ACL path.

Vectors must commit with order rows in one transaction. Which store fits?

Rule: Aurora PostgreSQL with pgvector, in the same account as the knowledge base.

An archive holds tens of millions of vectors, with monthly queries and a strict budget. Which store?

Rule: S3 Vectors as the store, because it is cheap for large, rarely queried sets.

A team names DynamoDB as the Knowledge Bases vector store. What is the issue?

Trap: DynamoDB is not on the Knowledge Bases store list. Keep it as the source.

Exact product codes are missed by meaning-only search. What search type fixes it?

Rule: hybrid search, with filterable keyword fields in the index.

Application code builds its own prompt from chunks. Which operation?

Rule: Retrieve, because your code receives the chunks and scores.

A question asks for total sales by region from rows. Which path?

Rule: structured retrieval to SQL, because it calculates over rows.

An OpenSearch filter on a custom field fails with a rewrite error. What is the mapping fix?

Rule: map the field as keyword, or as text with a keyword subfield.

A team picks a VPC-only managed domain for a Knowledge Base. What is the issue?

Trap: a Knowledge Base needs public network access on a managed OpenSearch domain.

Sources

Source: AWS documentation fetched for this module.

- https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/kb-chunking.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base-setup.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/rerank.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/kb-metadata.html
- https://docs.aws.amazon.com/AmazonS3/latest/userguide/s3-vectors.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/kb-advanced-parsing.html
- https://docs.aws.amazon.com/bedrock/latest/APIReference/API_agent-runtime_Retrieve.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/kb-data-source-sync-ingest.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base-build-structured.html

Source: course slide extract for this module, cited by page number in each section.